

Presenter Focus: Posture Geometry, Database-RAG, Docker, & CI/CD Pipelines

Total presentation time: 10 - 15 minutes. Your individual speaking time: ~5 - 7 minutes.

1. SLIDE SPEAKING SCRIPT (YOUR SECTIONS)

Slide 1: Introduction & System Handover

"Good morning, respected Principal, HOD, and members of the jury. Today, my teammate and I are presenting AlignMate - an AI-powered, real-time posture coach and fitness companion. While my teammate handles the client-side user interface HUD and WebSocket connection setup, I will guide you through the core backend engineering of the system: our posture geometry calculations, our structured database-driven RAG architecture for LLM personalization, our Dockerized containerization, and our automated CI/CD workflow."

Slide 4: Posture Geometry & Angle Calculations

"To track posture locally without uploading video, we analyze coordinate landmarks from MediaPipe. Specifically, I developed the vector calculations to detect neck tilt and shoulder slumping. First, we calculate the midpoint of the shoulders: $M_x = (Left_Shoulder_x + Right_Shoulder_x) / 2$ and $M_y = (Left_Shoulder_y + Right_Shoulder_y) / 2$. Then we draw a vector V pointing from this shoulder midpoint to the nose coordinate: $V = (Nose_x - M_x, Nose_y - M_y)$. We calculate the dot product of this vector with a vertical unit vector $(0, -1)$ to find the angle of the head relative to the vertical plane: $\cos(\theta) = -V_y / ||V||$. If the smoothed angle exceeds 15 degrees, the system flags a Lateral Neck Tilt. Similarly, if the vertical height difference between the left and right shoulders exceeds 0.03, we flag a Shoulder Imbalance."

Slide 5: Database-Driven RAG System (AI Planner)

"AlignMate features a custom workout and diet planner that operates on a structured database-driven RAG pattern. Instead of searching text chunks in a vector database, our system retrieves user telemetry directly from our MySQL database. When the user requests a plan, the backend queries the database for the user's physical profile (age, weight, height, lifestyle) and their historical posture records (average session score, frequent posture issues). This retrieved data is dynamically injected as context into a LangChain template and processed by Google Gemini (gemini-2.5-flash) to generate highly tailored fitness splits in a clean JSON format. This offloads model training while guaranteeing highly context-rich generation."

Slide 7: Docker Containerization

"To serve this application securely and uniformly on our DigitalOcean droplet, we containerized the stack using Docker. MediaPipe and OpenCV require specific C++ system rendering libraries (like GLib and GL) that are difficult to manage on standard host systems. Our Dockerfile packages these system dependencies into a lightweight python:3.10-slim image. Using Docker Compose, we run three services: Frontend (Nginx), Backend (FastAPI), and Database (MySQL). This keeps our MySQL database completely closed off from the public internet, accessible only within the internal bridge network, while exposing our web interfaces securely."

Slide 9: Technical Debugging & CI/CD Pipeline

"For quality assurance, we built a CI/CD pipeline using GitHub Actions. On every git push, the pipeline runs three concurrent jobs: First, backend-ci checks python compilation and runs pytest using an in-memory SQLite database. Second, frontend-ci tests the React production build to verify zero compilation errors. Third, docker-build-ci builds the backend and frontend Docker containers to ensure clean image builds. During development, we also resolved critical bottlenecks: we reduced WebSocket latency from 500ms to under 15ms by restricting payloads to bare landmark coordinates instead of image frames, and we resolved server OOM crashes by migrating from local Ollama to the cloud Gemini API."

2. LIVE DEMO RUNBOOK (YOUR ACTIONS)

Coordinate with your teammate to execute these actions smoothly.

- * **Action 1 - Live URL:** Point out that the application is fully live and hosted at <https://alignmate.tech>.
- * **Action 2 - Live HUD Analysis:** While your teammate stands in front of the camera, show the skeleton tracker overlay. Explain that as they tilt their head or shrug their shoulders, the backend is computing the vectors and angles in real-time. Slouch on purpose to trigger the voice coach ('Sit straight') and the shoulder roll recovery rep verifier.
- * **Action 3 - AI RAG Execution:** Navigate to the AI Planner. Click 'Generate Workout Plan'. Explain: 'Right now, the backend is querying our database for the user's historical telemetry, augmenting our prompt templates, and invoking Gemini to return this customized plan in real-time.'

3. JURY Q&A HANDBOOK (FOR YOUR SECTIONS)

- * **Q: Why did you use the Gemini API instead of running Ollama/Llama locally?:** Running LLMs locally on our cloud droplet requires 4-8GB of dedicated RAM and heavy CPU cycles, costing upwards of \$40/month. By migrating to the Gemini API, we reduced server RAM usage by 75%, allowing the backend to run on a cheap \$12/month droplet while increasing response speeds.
- * **Q: How do you handle user privacy?:** We never capture or upload raw video to our server. All video frame processing is done locally on the client's browser using MediaPipe. The browser only sends numerical float coordinates (joint vectors) over WebSockets, keeping the user's video 100% private.
- * **Q: Why did you choose WebSockets over normal HTTP REST requests?:** HTTP requests have high connection overhead (headers and handshakes) for every frame. Because we need to calculate posture angles at 15-20 frames per second, WebSockets allow us to maintain a single, lightweight, persistent connection with sub-15ms processing latency.